



National University of Computer and Emerging Sciences

Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab 1 – Introduction to Data Science

Instructor: Mariam Nasim

Introduction to Python

Objectives:

- Understand the basic environment of Jupyter Notebook.
- Write and execute Python code in cells.
- Use variables and operators effectively.
- Work with basic data types in Python.
- Implement control structures like if, for, and while.

Python Interpreters

1. Google Colab

Google Colab (short for Colaboratory) is a free, cloud-based Jupyter notebook environment provided by Google. It allows you to:

- Write and execute Python code.
- Use GPUs/TPUs for faster computations.
- Collaborate on projects in real time.
- Access powerful libraries without installation.

Step 1: Access Google Colab

1. **Sign in to Google Account:** Ensure you are logged into your Google account (sign in with your FAST LHR email)
2. **Open Google Colab:**
 - Go to: Google Colab.
 - Alternatively, you can access it via Google Drive:
 - Open Google Drive.
 - Click on "**New**" > "**More**" > "**Google Colaboratory**" to create a new notebook.

Step 2: Understand the Notebook Interface

The Google Colab interface is similar to Jupyter Notebook. Key components include:

1. **Cells:**
 - **Code Cell:** For writing and running Python code.
 - **Text Cell:** For writing markdown or plain text (e.g., instructions or explanations).
2. **Toolbar:**

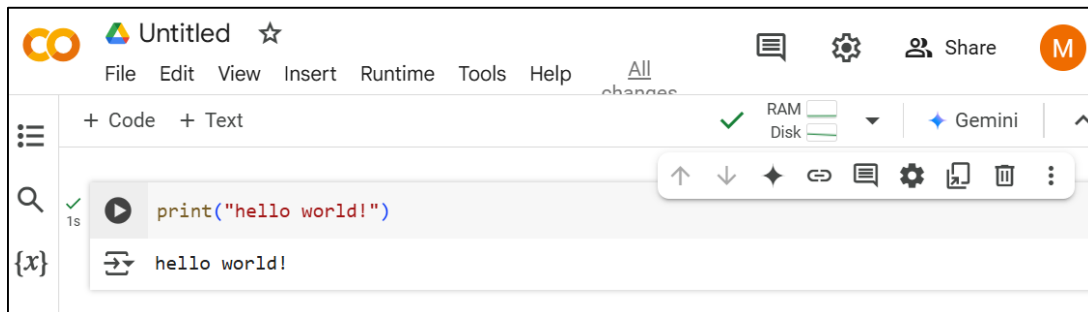
- **File:** To create, open, save, or download notebooks.
 - **Runtime:** To manage the computational backend (e.g., restart runtime, select GPU).
 - **Insert:** To add new cells (code or text).
3. **Sidebar:**
- Manage files and datasets in the notebook environment.

Step 3: Create a New Notebook

1. **Start a New Notebook:**
 - Open Colab and click on "**New Notebook**".
 - Alternatively, in Google Drive, go to "**New > More > Google Colaboratory**".
2. **Rename the Notebook:**
 - Click on the notebook's default name (**Untitled.ipynb**) in the top-left corner.
 - Provide a meaningful name for the notebook.

Step 4: Write and Run Code

1. **Add a Code Cell:**
 - Click on the "+ Code" button to add a new cell.
2. **Write Python Code:**
3. **Run the Code:**
 - Click the **Play** button on the left of the cell or press **Shift + Enter**.



2. Jupyter Notebooks:

Step 1: Download Anaconda

1. Visit the official Anaconda website.
2. Download the appropriate version for your operating system (Windows, macOS, or Linux).

- Choose the **Python 3.x** version.

Step 2: Install Anaconda

Step 3: Launch Jupyter Notebook via Anaconda Navigator

1. Open the **Anaconda Navigator** (search for it in your start menu or applications folder).
2. In Anaconda Navigator, you'll see an option for **Jupyter Notebook**.
3. Click the **Launch** button next to Jupyter Notebook. It will open in your default web browser.

Helping Material:

You can refer to the following links, if any details have been missed in the in manuals.

- a) Data Types
 - a. Built in types: https://www.w3schools.com/python/python_datatypes.asp
 - b. Typecasting: https://www.w3schools.com/python/python_casting.asp
- b) Operators https://www.w3schools.com/python/python_operators.asp
 - a. Math
 - b. Comparison
 - c. Boolean
- c) If-else conditions: https://www.w3schools.com/python/python_conditions.asp
- d) Loops: https://www.w3schools.com/python/python_for_loops.asp

https://www.w3schools.com/python/python_while_loops.asp

Variables

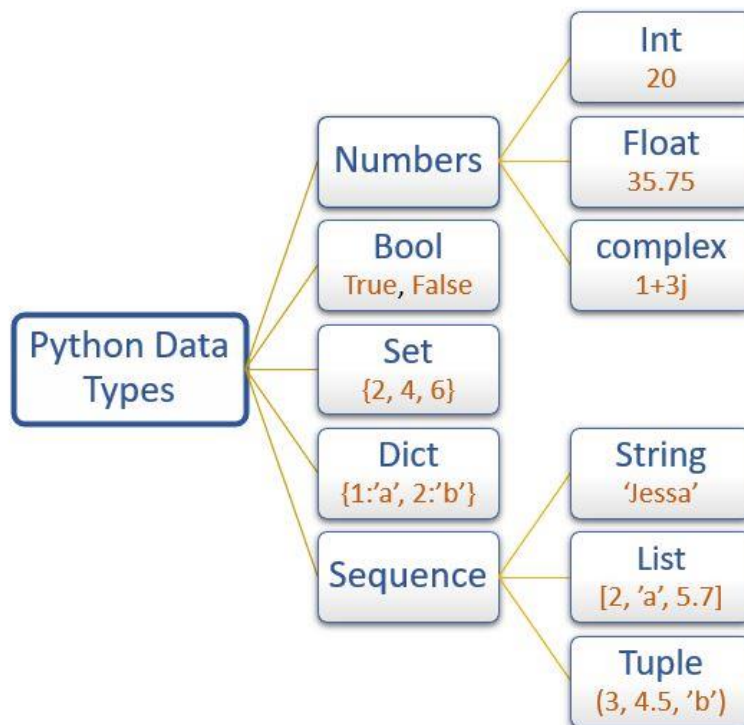
- Variables store data values.
- Python is dynamically typed, meaning no need to declare variable types.
- Names must start with a letter or underscore.
- Cannot use keywords (e.g., if, for, while).

```
x = 10
name = "Alice"
pi = 3.14
```

Python Data types:

Python has no command for declaring a variable for any datatype. A variable is created the moment you first assign a value to it. Variable names are casesensitive. Just like in other languages, Python allows you to assign values to multiple variables in one line. Basic data types are:

Data Type	Example	Description
<code>int</code>	<code>5</code> , <code>-3</code>	Whole numbers
<code>float</code>	<code>3.14</code> , <code>-2.0</code>	Decimal numbers
<code>str</code>	<code>"hello"</code>	Text (string)
<code>bool</code>	<code>True</code> , <code>False</code>	Boolean (logic)
<code>list</code>	<code>[1, 2, 3]</code>	Ordered collection



The process of explicitly converting the value of one data type (int, str, float, etc.) to another data type is called type casting. In Type Casting, loss of data may occur as we enforce the object to a specific data type.

Example

Integers:

```
x = int(1)    # x will be 1
y = int(2.8)  # y will be 2
z = int("3")  # z will be 3
```

Operators

This section contains the details of different Python operators i.e. Math operators, comparison operators and Boolean operators.

Arithmetic operators are used with numeric values to perform common mathematical operations.

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

Operator Precedence:

Operator	Description
<code>()</code>	Parentheses
<code>**</code>	Exponentiation
<code>+X</code> <code>-X</code> <code>~X</code>	Unary plus, unary minus, and bitwise NOT
<code>*</code> <code>/</code> <code>//</code> <code>%</code>	Multiplication, division, floor division, and modulus
<code>+</code> <code>-</code>	Addition and subtraction
<code><<</code> <code>>></code>	Bitwise left and right shifts
<code>&</code>	Bitwise AND
<code>^</code>	Bitwise XOR
<code> </code>	Bitwise OR
<code>==</code> <code>!=</code> <code>></code> <code>>=</code> <code><</code> <code><=</code> <code>is</code> <code>is not</code> <code>in</code> <code>not in</code>	Comparisons, identity, and membership operators
<code>not</code>	Logical NOT
<code>and</code>	AND
<code>or</code>	OR

Control Structures:

1) If-else Statement

Python supports conditional statements i.e. `if`, `elif`, `else`. Comparison operators and Boolean operators written in the previous section can be used in `if-elif-else` statements.

An "if statement" is written by using the `if` keyword.

Python relies on indentation (whitespace at the beginning of a line) to define scope in the code. Other programming languages often use curly-brackets for this purpose.

If statement:

```
a = 33
b = 200
if b > a:
    print("b is greater than a")
```

The `elif` keyword is Python's way of saying "if the previous conditions were not true, then try this condition".

The `else` keyword catches anything which isn't caught by the preceding conditions.

Example

```
a = 200
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
else:
    print("a is greater than b")
```

Python conditional statements only require `if` statement to execute. Both `elif` and `else` are optional and used as per requirement.

2) While Loop:

■ While loop

- Iterate while the specified condition is True

```
In [1]: counter = 0
        while counter < 5:
            print (f"The value of counter is {counter}")
            counter += 2    # increment counter of 2
```

```
Out [1]: The value of counter is 0
          The value of counter is 2
          The value of counter is 4
```

With the `break` statement we can stop the loop even if the while condition is true:

Example

Exit the loop when i is 3:

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

With the `continue` statement we can stop the current iteration, and continue with the next:

Example

Continue to the next iteration if i is 3:

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

3) While Loop:

A `for` loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

With the `for` loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Example

Print each fruit in a fruit list:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
```

Loop through the letters in the word "banana":

```
for x in "banana":
    print(x)
```

With the `break` statement we can stop the loop before it has looped through all the items:

Example

Exit the loop when `x` is "banana":

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
    if x == "banana":
        break
```

With the `continue` statement we can stop the current iteration of the loop, and continue with the next:

Example

Do not print banana:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

To loop through a set of code a specified number of times, we can use the `range()` function,

The `range()` function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Example

Using the start parameter:

```
for x in range(2, 6):
    print(x)
```

Lab Tasks: (marks = 25)

Task 1: Declare a variable radius and assign it a value (e.g., radius = 5). Then calculate and print the area of a circle using the formula:

$$\text{Area} = \pi \times r^2$$

Print the area up to **2 decimal places**. (3)

Task 2: Repeat Task 1 by asking the user to enter the radius of a circle. Convert the input into a float, calculate the area using the same formula, and print the result up to 2 decimal places. (3)

Task 3: Ask the user for their age and print whether they are eligible to vote (18 or older). (3)

Task 4: Write a Python program that accepts a year from the user and checks whether it is a **leap year** or not. (4)

A year is considered a leap year if it meets the following conditions:

- It is divisible by 4 **and** not divisible by 100, **or**
- It is divisible by 400

Task 5: Use a while loop to print even numbers from 2 to 20. (2)

Task 6: Ask the user to enter a number and use a for loop to calculate its factorial. (2)

Task 7: Write a Python program that: (3)

1. Asks the user to enter a temperature **value** and the **unit** they want to convert from (C for Celsius, F for Fahrenheit).
2. Converts the temperature to the other unit:
 - Celsius to Fahrenheit: $F = (C \times 9/5) + 32$
 - Fahrenheit to Celsius: $C = (F - 32) \times 5/9$
3. Displays the converted temperature.

Task 8: Write a program that takes a positive integer as input. Checks if the number is an Armstrong number. Prints an appropriate message. A number is called an Armstrong number if the sum of the cubes of its digits is equal to the number itself. (5)

$$153 \rightarrow 1^3 + 5^3 + 3^3 = 153 \quad \checkmark$$

$$370 \rightarrow 3^3 + 7^3 + 0^3 = 370 \quad \checkmark$$

$$123 \rightarrow 1^3 + 2^3 + 3^3 = 36 \quad \times$$

(Do not use any built-in function for this lab)

Instructions:

- Submit a **single .ipynb** file. Any other extension would be marked zero.
- **Rename** your file to “rollnum_Name.ipynb” before submitted. Files with no names will be marked zero.
- Any submission **after deadline**, will be marked zero.
- Do not copy from each other / any AI tool.